Paso 18 — CSCV y PBO (probabilidad de sobreajuste): qué se calcula, con qué estadística y qué criterio hay cargado

Fecha: 2026-10-07 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 18 (`studies/optimisation/cscv/`). Del paso 16.5 (la fábrica de variantes, `sqx/variants/`) se explica lo justo para entender qué lee el CSCV. · **Revisado** por dos miembros del Council (Feynman, Taleb) el 2026-10-07; correcciones aplicadas.

Qué es: el documento de consulta del paso 18, con la misma forma que el del paso 16 (`criterios-paso-16-spp-2026-10-07`). Explica desde cero la estadística que este paso añade, repasa **todos** los números que escribe el estudio, cita el criterio tal como está hoy (ninguna regla en `pipeline/autopilot/criteria.yaml`; un único umbral, dentro del propio estudio) y cierra con lo que no cuadra y con las decisiones que te quedan.

De dónde salen los números. Todos los ejemplos son de la corrida `Trade_NIKKEI225_M30_volatilidadL` del 2026-10-04 (NIKKEI225, M30, largas): diez estrategias madre, entre 276 y 501 variantes de cada una, 3.722 días de curva por variante. Están **recalculados por mí desde disco** (`AlgoData/strategyPermutations/Trade_NIKKEI225_M30_volatilidadL/Strategy_<x>/equity.parquet` y `metrics.parquet`) con las funciones del estudio y comparados con el `cscv.json` que el estudio dejó escrito aquel día: el PBO coincide hasta el cuarto decimal en las diez, y también el DSR, el listón de Sharpe, el percentil OOS y la dominancia. Lo que el estudio no escribe —el PBO con otro número de bloques, sin controles, con otro suelo, la pendiente del artículo, y la distribución del PBO cuando no hay nada que elegir— lo he calculado yo desde los mismos ficheros y lo digo en cada sitio. «Ilustrativo» quiere decir cuenta hecha a mano o simulación para enseñar, no medida.

Ninguna línea de este documento está congelada. El fichero de criterios lo dice en su comentario de `steps` (`pipeline/autopilot/criteria.yaml:59`): provisional hasta que una población sin ventaja haya recorrido la cadena, y los pasos 17, 18, 19, 21, 23 y 24 «sólo anotan: sin regla a propósito».

---

0. En una página

La pregunta del paso. De una estrategia hay unas 300-400 variantes con los parámetros movidos. Si eliges la variante que mejor fue en una mitad del histórico, ¿sigue siendo de las buenas en la otra mitad, o haber sido la mejor fue suerte? Si es suerte, «optimizar los parámetros» de esa estrategia no sirve y lo que enseña el backtest del mejor punto está inflado.

Lo que hay cargado hoy: ninguna regla en `criteria.yaml` (no existe la entrada `"18"`). El paso es parte del bloque ciego 17 / 18 / 18.5 / 19, que se calcula sin leerse. El único umbral vive dentro del estudio: **PBO de la regla «Argmax» por encima de 0,5 = `fail`; si no, `pass`** (`studies/optimisation/cscv/contract.py:87`). Ese estado lo recoge el paso 20 (`blindJoint`) con la lectura `sin_fallo`: «ninguna pieza del bloque en `fail`». Un `fail` del CSCV marca DESCARTAR en el `verdict.csv` del paso 20 (`blindJoint/report.py:64`), pero el autopiloto no lo aplica y todas siguen; sólo corta si se aplica a mano con `/curate`.

Lo que sale en la corrida real: 8 `pass` y 2 `fail` (`13.3.390`, PBO 0,642, y E `3.11.426`, PBO 0,653). En el paso 20 las diez salen DESCARTAR, pero no por el CSCV: `marketSurfaces` (18.5) falla en las diez. Y el autopiloto no corta nada en el 20, porque tampoco tiene reglas.

Las ocho cosas que conviene saber antes de leer el resto:

1. **El PBO no mide si la estrategia es buena. Mide si el orden entre sus variantes se conserva de una mitad del histórico a la otra.** Una meseta perfecta —todas las variantes igual de buenas, que es lo que tú llamas robusta— da PBO alrededor de 0,5, porque entre iguales el «mejor» de una mitad es un sorteo. Lo he medido sobre los paneles reales (apartado 2.12): con todas las variantes igualadas al mismo Sharpe, el PBO sale 0,50 de media **con desviación típica 0,19**, y supera 0,5 en el **51 %** de los sorteos (46-56 % según la madre). Es decir: el umbral cargado, «`fail` si PBO > 0,5», suspende a cara o cruz a una estrategia cuyas variantes no se distinguen entre sí. El revisor lo confirmó con una simulación propia: el centro en 0,5 es estructural (no lo mueven la dependencia de un día con el siguiente, los días planos, la correlación entre variantes ni los efectos de época); la sd es de cada panel, entre 0,14 y 0,21. Lo contrario no vale: **un PBO bajo no niega la meseta.** El PBO mira el orden, no el tamaño de las diferencias; un orden estable con diferencias minúsculas también da PBO bajo (apartado 2.12).
2. **Un PBO sólo se distingue del azar por debajo de 0,18, más o menos.** Ese es el percentil 5 de la distribución anterior (0,184 ± 0,02). De las diez, quedan por debajo **dos**: G `4.13.338` (0,073; p < 0,02 **sin corregir**) y `3.9.284` (0,140; **indicio**, sin nula propia). Son las dos mejores de diez: que dos o más de diez caigan bajo el percentil 5 por puro azar ocurre el 8,6 % de las veces, y más si las madres comparten histórico. Los dos `fail` (0,642 y 0,653) y los seis `pass` entre 0,29 y 0,45 **no rechazan la nula**; tampoco la confirman, porque la prueba tiene poca potencia en esa zona: con una diferencia verdadera de Sharpe de 0,05 anual entre variantes, el PBO medio es 0,32 y sólo el 8 % de las veces baja de 0,18 (simulación del revisor). Son compatibles con «variantes iguales» y con un orden real modesto.
3. **Las celdas de la matriz no son la ganancia de cada día.** Son la diferencia, de un día al siguiente, del **mínimo** de la curva de capital de ese día (`knowhow/sqx-format/daily-equity-bin.md`, probado el 2026-10-04). El `config.yaml` del estudio dice que son curvas «marcadas a mercado» y que «el 75 % de las celdas se mueve»; aquí se mueve entre el 14 % y el 50 %. Cuánto cambia el PBO de un lote de variantes por esto, no está medido.
4. **El PBO depende del reparto IS/OOS elegido y del suelo de operaciones, aunque el código diga que no.** `engines/variants/config.yaml:40-42` afirma que «el PBO es el MISMO número en todas las composiciones». El estudio descarta antes las variantes con menos de 50 operaciones en `oos2`, y eso quita entre el 9 % y el 36 % del panel. C `14.3.479` pasa de 0,292 a 0,200 con el otro reparto; `13.3.390` pasa de 0,642 (`fail`) a 0,423 (`pass`) sin suelo.
5. **La segunda regla, «Centro del plateau», es casi la misma que «Argmax».** Con radio 1, entre el 21 % y el 100 % de las variantes no tiene ningún vecino, y para ellas la regla es idéntica a elegir el máximo. En `13.3.390` las dos reglas eligen lo mismo en las 12.870 particiones. Donde difieren, el plateau sale igual o **peor** en 8 de 10, aunque sólo en tres (`16.5.352`, `18.5.355`, `3.9.284`) la diferencia supera el ruido.
6. **Tres de los números que escribe el estudio no informan.** La dominancia es «ninguna» en las diez estrategias y con las dos reglas, por construcción (basta una partición mala). La «pendiente» guardada es una extensión del proyecto, no la regresión del artículo, y tiene el signo contrario (la del artículo sale

negativa en las diez y sólo se dibuja). El intervalo del «percentil OOS» va de 5 a 93 en varias: una sola partición cronológica no da para más.

7. **Los controles compiten como candidatos.** En el panel entran el origen (la madre, `P00000`), los canarios y las parejas inertes del paso 16.5, y columnas duplicadas. En `19.2.363` la «elegida» es un canario en el 20 % de las particiones; sin canarios su PBO pasa de 0,391 a 0,441. En G, sin columnas duplicadas, de 0,073 a 0,121. Son movimientos de 0,03-0,05, un cuarto de la sd nula: la aritmética es cierta, el efecto no está demostrado.
8. **Ningún número del CSCV llega a `hechos.parquet`, y una regla `"18"` no se juzgaría aunque la escribieras.** El extractor de hechos sólo acepta resultados que sean carpeta y el del CSCV es un fichero (`pipeline/autopilot/facts.py:221-222`); el veredicto deja `score` vacío a propósito (`contract.py:81-82`); y la cola del autopiloto apunta los pasos del bloque como «ciegos» sin pasarlos por el juez (`pipeline/autopilot/tail.py:110`). Hoy el único camino por el que el CSCV deja huella es su `pass`/`fail` dentro del paso 20, y ahí marca sin cortar.

Además, y aparte del PBO: el **Sharpe deflactado (DSR)** que el estudio escribe se calcula sobre `build` + `oos1`, es decir, con el tramo de fabricación dentro. Tu doctrina es que ese tipo de lectura (PSR, trader al azar) sólo se hace fuera de muestra. Está en el apartado 5.10.

---

1. Qué es el paso 18 y cómo decide

1.1 Dónde está

Familia «optimización» (`studies/optimisation/`). Viene después del SPP (15-16), que mueve los parámetros dentro de SQX, y de la fábrica de variantes (16.5), que fabrica unas 500 versiones de cada estrategia y las vuelve a probar en tres tramos. Sobre esas variantes trabajan cuatro estudios: la nube de parámetros, el WFC (17), el CSCV (18) y las superficies por mercado (18.5). La WFM (19) vuelve a SQX.

Los pasos 17, 18, 18.5 y 19 forman el **bloque ciego**: se calculan para todas las madres que llegan, nadie mira el resultado de uno para decidir si corre el siguiente, y se leen juntos en el paso 20 (`pipeline/autopilot/blind.py:23-48`, «computed and not read»). Es tu regla de que del 16 al final todas las supervivientes pasan por los cuatro.

1.2 Lo que deja el paso 16.5, en lo que toca al CSCV

Por cada madre, un lote en `AlgoData/strategyPermutations/<proyecto>/Strategy_<x>/`. Lo que importa aquí:

Cuántas variantes y cuáles. El tope es 500 por madre (`n_target: 500`, `studies/breakage/spp/config.yaml:18`). Es tope y no cuota (`sqx/variants/design/plan.py:57`): no se repiten combinaciones para llegar a 500. Las partes de cada estrato son del tope (`config.yaml:19-22`):

estrato	parte	qué son
vecindario	20 %	la madre con los parámetros movidos poco, en pasos de rejilla
factorial	60 %	combinaciones de unos pocos niveles por parámetro
cobertura	20 %	puntos repartidos por todo el rango (secuencia de Sobol), con valores que no caen en la rejilla

Y unos **controles** que no son candidatos de verdad: el **origen** `P00000` (la madre tal cual), los **canarios** (hasta tres permutaciones del SPP elegidas a propósito: la de mayor beneficio, la de menor y la de menos operaciones, sin repetir, y sólo si la tabla del SPP tiene alguna permutación con los desplazamientos de la madre; `sqx/variants/design/canaries.py:22-24` y `:64-66`) y las **parejas inertes** (una variante por cada parámetro congelado: la madre con ese parámetro puesto en el valor más lejano de su rango; se compara con el origen y se espera el mismo resultado; `canaries.py:76-83`). Sirven para comprobar que la fábrica funciona. El CSCV no los aparta (apartado 5.7).

Tres tramos, probados por separado. Cada variante se vuelve a probar en `build`, `oos1` y `oos2` como tres tareas distintas (`WFC_Build`, `WFC_OOS1`, `WFC_OOS2`), en un proyecto propio, nunca en el `Retester` de fábrica. De cada tramo se guarda la curva diaria; las tres se pegan una detrás de otra en `equity.parquet`.

Un suelo duro al recoger. Las variantes con menos de 100 operaciones en total (sumando los tres tramos) no entran en `metrics.parquet` (`assets/_build.yaml:368`, `wfc.min_trades_total: 100`).

Qué hay en cada celda de `equity.parquet`. Una fila por día, una columna por variante, y en la celda **el incremento de la curva de ese día**. La curva que SQX guarda no es el capital al cierre: es el **capital más bajo** que se vio ese día mientras había posición abierta. Así que la celda es «mínimo de hoy menos mínimo de ayer». Sumadas a lo largo de los años dan aproximadamente el beneficio total; día a día no son la ganancia del día. Un día plano a continuación de una operación recoge el salto del mínimo al cierre previo; los días planos siguientes valen cero. La suma telescopa al último valor de la curva (≈ beneficio neto, salvo el último día, que se tira); la ficha lo cuadra con el IS a 0,59 USD (`knowhow/sqx-format/daily-equity-bin.md:11-15` y `:63`).

Los números de línea de `sqx/variants/` de este apartado los ha comprobado el revisor (Feynman).

1.3 Qué datos lee el paso 18

Una matriz por madre: **T filas (días) × N columnas (variantes)**. El estudio la monta así (`studies/optimisation/cscv/report.py:130-136`, `engines/variants/panel.py`):

1. Lee `equity.parquet`, suma las filas de la misma fecha y tira la última (`panel.py:116-117`). En esta corrida: 3.768 filas, 45 fechas repetidas (cada tramo arranca unos dos meses antes de su fecha, con P&L cero, y se solapa con el final del anterior), 3.723 fechas distintas, **3.722 días** de 2011-09-19 a 2026-09-28.
2. Se queda con las variantes que tienen **al menos 50 operaciones a cada lado del reparto cronológico** (`panel.py:158-159`, `min_trades: 50` en `engines/variants/config.yaml:19`). El reparto cargado es `oos2_only`: dentro = `build` + `oos1`, fuera = `oos2` (`config.yaml:46`). En la práctica el suelo que muerde es el de `oos2`: ninguna variante falla por el lado de dentro.

Lo que queda, madre a madre:

madre	variantes en `metrics`	fuera por < 50 op. en `oos2`	N que entra	columnas distintas	celdas que no son cero
`10.11.484`	420	99	**321**	276	29 %
`13.3.390`	276	61	**215**	215	50 %
C `14.3.479`	417	100	**317**	253	26 %
F `15.9.352`	501	71	**430**	422	34 %
`16.5.352`	499	61	**438**	438	43 %
`18.5.355`	376	111	**265**	265	40 %
`19.2.363`	402	50	**352**	352	14 %
E `3.11.426`	429	156	**273**	270	23 %
`3.9.284`	416	89	**327**	327	34 %
G `4.13.338`	460	43	**417**	281	31 %

«Columnas distintas» cuenta curvas que no son idénticas día a día a otra. En G, 136 de las 417 variantes son copia exacta de otra: parámetros distintos, mismas operaciones.

1.4 Dos ventanas distintas dentro del mismo estudio

El estudio hace dos cosas con el mismo panel, y conviene no mezclarlas:

	qué histórico usa	para qué
El CSCV propiamente dicho	los 3.722 días enteros, `build` + `oos1` + `oos2`, cortados en 16 bloques	PBO, ω, «pierde», pendiente, dominancia
El bloque cronológico	dentro = `build` + `oos1` (3.014 días); fuera = `oos2` (708 días)	DSR, grupos de variantes, percentil OOS, deriva

El CSCV lee `oos2`. Lo declara (`report.py:26`, `READS = panel.SEGMENTS`) y `assets/_policy.yaml` se lo concede desde el 2026-09-27. Cada ejecución del paso apunta una fila por tramo en el libro de miradas (`report.py:182-188`). Los 16 bloques, con sus fechas:

bloque	fechas	tramo
1	2011-09-19 → 2012-12-24	`build`
2-8	hasta 2019-07-18	`build`
9	2019-07-19 → 2020-06-12	a caballo entre `build` y `oos1`
10-12	hasta 2023-02-22	`oos1`
13	2023-02-23 → 2024-01-17	a caballo entre `oos1` y `oos2`
14-16	hasta 2026-09-28	`oos2`

Diez bloques tienen 233 días y seis tienen 232 (unos once meses cada uno). El primero abarca quince meses de calendario porque incluye el arranque en vacío.

1.5 Los estados que da

Dos, no tres: **`pass`** o **`fail`**. No hay `watch` ni Limbo. La frontera es PBO de la primera regla de la lista (`Argmax`) mayor que 0,5 (`contract.py:87`). Un PBO de exactamente 0,5 es `pass`.

Cada regla de selección lleva además su propia etiqueta `pass` / `fail` con la misma frontera (`contract.py:95`), pero sólo la primera decide el estado del estudio.

1.6 Cómo llega eso a una decisión

No llega por `criteria.yaml`. Llega por el paso 20:

1. `studies/closing/blindJoint/` lee el estado de cada pieza del bloque: `wfc`, `cscv`, `marketSurfaces`, `wfm` (`pieces.py:12`, `said` en `:56-60`).
2. La lectura cargada es `pieces: sin_fallo` (`blindJoint/config.yaml:26`, tuya del 2026-10-04): la madre pasa si **ninguna pieza está en `fail`** (`readings.py:17`).
3. Si no pasa, `blindJoint` escribe DESCARTAR en su `verdict.csv` (`report.py:64`).
4. El autopiloto mira el paso 20 con sus reglas; como no hay entrada `"20"` en `criteria.yaml`, el juez responde «ninguno: el paso no tiene criterio» y no corta (`pipeline/autopilot/judge.py:169-173`).

Así que hoy el `fail` del CSCV **marca** pero no **borra**: marca DESCARTAR en el `verdict.csv` del paso 20 (`blindJoint/report.py:64`), pero el autopiloto no lo aplica y todas siguen; sólo corta si se aplica a mano con `/curate`. El paso 20 no tiene `cut` y `blindJoint` no llama a `apply_verdict`.

1.7 Cómo se lanza y qué no he podido medir

El autopiloto lo llama con `python3 -m studies.optimisation.cscv.report --work <lote> --family <familia>`, sin `--blocks` y sin cambiar nada de la configuración (`ui/daemon/runner/optimisation.py:30-47`, lectura delegada). Todo lo demás sale de `studies/optimisation/cscv/config.yaml` y de `ledger/thresholds.yaml`.

Cuánto tarda el paso por madre en la corrida real no lo he medido (no he lanzado el estudio: sólo he llamado a sus funciones sobre los ficheros).

1.8 Las estrategias que nos acompañan

Las mismas cuatro del dossier del paso 16, con las mismas letras:

	madre	N	PBO Argmax	estado	en una línea
C	`14.3.479`	317	0,292	pass	la del medio; sirve para las cuentas a mano
E	`3.11.426`	273	0,653	fail	una de las dos que suspenden
F	`15.9.352`	430	0,350	pass	pasa, y su elegida pierde dinero en el 55 % de las particiones
G	`4.13.338`	417	0,073	pass	la única por debajo de todos sus nulos (p < 0,02 sin corregir)

---

2. Fundamentos: la estadística de este paso, desde cero

2.0 Lo que ya sabes de los dossiers anteriores

- **Sharpe:** media de las ganancias dividida por su desviación típica. Aquí se calcula sobre celdas diarias; para leerlo en años se multiplica por √252 ≈ 15,87. Un Sharpe diario de 0,045 es 0,72 anual.
- **Mediana y percentil:** la mediana es el valor que deja la mitad por debajo; el percentil 5, el que deja el 5 %.
- **Desviación típica (sd):** cuánto se separan los valores de su media, en las mismas unidades.
- **Distribución nula:** lo que saldría si no hubiera nada que encontrar. Un número sólo dice algo si cae en la cola de su nula.

2.1 El problema: elegir el mejor infla el resultado

Tira diez monedas veinte veces cada una. Alguna sacará 14 caras. Si la eliges «por ser la mejor» y la vuelves a tirar veinte veces, sacará unas 10. No era mejor: fue la que tuvo más suerte, y al elegirla te llevaste su suerte como si fuera mérito.

Con parámetros pasa lo mismo. De 317 variantes, la de mejor Sharpe en el backtest lo es en parte por ser buena y en parte por suerte. La pregunta del CSCV es **cuánto de cada cosa**: si repitiéramos la elección en un trozo de histórico y la comprobáramos en otro, ¿la elegida seguiría arriba?

Con una sola partición (un tramo para elegir, otro para comprobar) obtienes una respuesta, y una respuesta no dice si fue casualidad. El CSCV hace **muchas** particiones del mismo histórico.

2.2 La matriz y los bloques

Un ejemplo que cabe en la mano: **4 variantes** (A, B, C, D) y **8 días**. En cada celda, lo que ganó esa variante ese día.

día	A	B	C	D
1	0	6	3	1
2	0	5	-2	3
3	3	1	0	2
4	1	1	2	-1
5	3	0	-1	2
6	2	0	1	-1
7	0	-2	1	-2
8	1	1	-2	0

Con el histórico entero, el Sharpe de cada una es A 0,98 · B 0,56 · C 0,14 · D 0,28. A parece la mejor.

Se corta el histórico en **S bloques seguidos de igual tamaño**. Aquí S = 4, de dos días cada uno: bloque 1 = días 1-2, bloque 2 = días 3-4, bloque 3 = días 5-6, bloque 4 = días 7-8. Los días de un bloque nunca se separan: así se respeta que los días cercanos se parecen.

En el estudio S = 16 (`ledger/thresholds.yaml:69-75`, tuyo del 2026-09-30; antes 12). Si T no es divisible entre S, los bloques no salen iguales: el código reparte el resto (`np.array_split`, `measure/cscv.py:89-93`) y aquí hay diez de 233 días y seis de 232. El artículo original, hasta donde recuerdo, recorta el histórico para que sean iguales (no verificado contra el artículo; el revisor tampoco lo tenía delante). Un día de diferencia sobre 233 no cambia nada.

2.3 Las combinaciones: de cuántas maneras se reparten los bloques

Se reparten los S bloques en dos mitades: S/2 bloques para **elegir** («dentro», IS) y los otros S/2 para **comprobar** («fuera», OOS). Y se hace **de todas las maneras posibles**.

Con 4 bloques hay que escoger 2 para «dentro». Las maneras:

```
dentro {1,2}  fuera {3,4}
dentro {1,3}  fuera {2,4}
dentro {1,4}  fuera {2,3}
dentro {2,3}  fuera {1,4}
dentro {2,4}  fuera {1,3}
dentro {3,4}  fuera {1,2}
```

Seis. El número se escribe C(4,2) y se lee «combinaciones de 4 tomadas de 2 en 2». La cuenta general:

```
C(S, S/2) = S × (S−1) × … × (S/2 + 1)  /  (S/2 × (S/2 − 1) × … × 1)

C(4,2)  = (4 × 3) / (2 × 1)                          = 6
C(16,8) = (16×15×14×13×12×11×10×9) / (8×7×6×5×4×3×2×1) = 12.870
```

En el código es `itertools.combinations(range(blocks), blocks // 2)` (`measure/cscv.py:24`). Con 16 bloques, **12.870 particiones**; el `cscv.json` lo guarda en `partitions`. ✓

Tres cosas que salen de esta forma de repartir:

- **Es simétrica.** La partición {1,2}/{3,4} y la {3,4}/{1,2} están las dos: cada reparto se usa una vez en cada sentido. De ahí la «S» de CSCV (*symmetric*).
- **«Dentro» y «fuera» no son pasado y futuro.** En la mayoría de las particiones, «fuera» incluye bloques anteriores a algunos de «dentro». El CSCV no simula operar hacia delante; pregunta si lo que distingue a las variantes es estable en el tiempo.
- **`oos2` está en las dos mitades.** Los bloques 14, 15 y 16 son `oos2` puro. Sólo en 1.287 de las 12.870 particiones (C(13,8), el 10 %) los tres caen del lado de «fuera». En el 90 % restante, la elección ya ha visto parte de `oos2`.

2.4 El Sharpe de cada mitad

En cada partición se calcula el Sharpe de cada variante **dentro** y **fuera**, sobre los días de sus bloques puestos uno tras otro.

Para no repetir 12.870 veces la cuenta día a día, el código guarda de cada bloque y cada variante tres sumas —la suma de las celdas, la suma de sus cuadrados y la de los cuadrados de las negativas— y el Sharpe de cualquier mitad sale de sumar las de sus bloques (`measure/cscv.py:89-101`). Con el ejemplo, variante B, bloques 1 y 2 (días 1 a 4: 6, 5, 1, 1):

```
suma            = 6 + 5 + 1 + 1          = 13
suma cuadrados  = 36 + 25 + 1 + 1        = 63
media           = 13 / 4                 = 3,25
varianza        = (63 − 13 × 3,25) / 3   = 20,75 / 3 = 6,917
desviación      = √6,917                 = 2,630
Sharpe          = 3,25 / 2,630           = 1,236
```

La varianza divide entre n − 1 (`ddof=1`, `measure/cscv.py:38-40`). Si una variante no se mueve en toda una mitad (desviación cero), su Sharpe se pone a cero.

El estudio sabe calcular también el Sortino (sólo cuenta la dispersión de los días malos; `measure/cscv.py:61-67`), pero el cargado es `score: sharpe` (`cscv/config.yaml`).

2.5 Elegir, y el rango relativo ω

En cada partición:

1. Se **elige** la variante con mejor Sharpe **dentro** (`measure/cscv.py:185`). Es la regla «Argmax».
2. Se mira **en qué puesto queda esa misma variante fuera**, entre las N.

El puesto se expresa como una fracción entre 0 y 1, el **rango relativo ω** (omega):

```
ω = (nº de variantes con Sharpe fuera ESTRICTAMENTE menor que el de la elegida + 1) / (N + 1)
```

(`measure/cscv.py:188`.) ω cerca de 1: la elegida también es de las mejores fuera. ω = 0,5: queda en el medio. ω cerca de 0: de las peores.

El «+1» arriba y abajo evita que ω valga exactamente 0 o 1. El «estrictamente menor» hace que los empates jueguen en contra: si la elegida empata con otras fuera, ninguna de ellas cuenta como «por debajo». En los datos reales esto apenas pesa (he probado a repartir los empates a medias y el PBO se mueve menos de 0,001).

Con el ejemplo, partición dentro {1,2} / fuera {3,4}:

```
Sharpe dentro:  A 0,71   B 1,24   C 0,34   D 0,73     → elegida: B
Sharpe fuera:   A 1,16   B −0,20  C −0,17  D −0,15
variantes por debajo de B fuera: ninguna
ω = (0 + 1) / (4 + 1) = 0,2
```

B fue la mejor en los días 1-4 y la peor en los días 5-8.

2.6 El logit λ

Se transforma ω con una función que se llama **logit**:

```
λ = ln( ω / (1 − ω) )
```

`ln` es el logaritmo natural. (`measure/cscv.py:191`.) Lo que hace: estira la escala de 0 a 1 hasta una que va de menos infinito a más infinito, con el cero en el medio.

ω	ω / (1 − ω)	λ	lectura
0,2	0,25	−1,386	la elegida queda abajo
0,4	0,667	−0,405	algo por debajo del medio
0,5	1	0	justo en el medio
0,6	1,5	+0,405	algo por encima
0,8	4	+1,386	arriba

Lo único que importa para el PBO es el **signo**: λ positivo, la elegida quedó en la mitad de arriba; λ negativo o cero, en la de abajo. El valor exacto sólo se usa para dibujar el histograma.

Con 4 variantes ω sólo puede valer 0,2 · 0,4 · 0,6 · 0,8 y nunca cae en 0,5. Con N impar sí: la variante que queda justo en el centro tiene ω = 0,5 y λ = 0, y el código la cuenta como sobreajustada (siguiente apartado). Con N de 215 a 438 es una partición entre miles.

Un ejemplo real. C `14.3.479`, primera partición (dentro = bloques 1 a 8, de 2011-09 a 2019-07; fuera = bloques 9 a 16):

```
elegida dentro:  P00096, Sharpe diario 0,0600  (0,95 anual)
la misma fuera:  Sharpe diario 0,0238          (0,38 anual)
variantes por debajo de ella fuera: 268 de 317
ω = (268 + 1) / (317 + 1) = 269 / 318 = 0,846
λ = ln(0,846 / 0,154) = ln(5,49) = 1,70
```

En esa partición la elegida perdió más de la mitad de su Sharpe y aun así quedó por delante del 85 % de las variantes. Las dos cosas a la vez: es lo normal.

2.7 El PBO

PBO (*Probability of Backtest Overfitting*): la fracción de las particiones en las que la elegida quedó en la mitad de abajo.

```
PBO = (nº de particiones con λ ≤ 0) / (nº de particiones)
```

(`verdict/summary.py:21`, `(lam <= 0).mean()`.)

Cómo se lee, según el artículo: PBO = 0 quiere decir que elegir dentro siempre acierta fuera; PBO = 0,5, que elegir dentro vale lo mismo que elegir a ciegas; PBO cerca de 1, que el mejor de dentro tiende a ser de los peores de fuera.

Lo que el PBO no es. No es la probabilidad de que la estrategia pierda. No es la probabilidad de que la estrategia esté sobreajustada. Es una propiedad del **procedimiento de elegir la mejor variante dentro de este conjunto concreto**. Cambia el conjunto (apartados 5.2 y 5.7) y cambia el PBO.

2.8 El ejemplo completo: las seis particiones

dentro	Sharpe dentro (A, B, C, D)	Sharpe fuera (A, B, C, D)	elegida	por debajo fuera	ω	λ
{1,2}	0,71 · **1,24** · 0,34 · 0,73	1,16 · −0,20 · −0,17 · −0,15	B	0	0,2	−1,386
{1,3}	0,83 · **0,86** · 0,11 · 0,73	0,99 · 0,17 · 0,15 · −0,15	B	2	0,6	+0,405
{1,4}	0,50 · **0,68** · 0,00 · 0,24	2,35 · 0,87 · 0,39 · 0,29	B	2	0,6	+0,405
{2,3}	**2,35** · 0,87 · 0,39 · 0,29	0,50 · 0,68 · 0,00 · 0,24	A	2	0,6	+0,405
{2,4}	**0,99** · 0,17 · 0,15 · −0,15	0,83 · 0,86 · 0,11 · 0,73	A	2	0,6	+0,405
{3,4}	**1,16** · −0,20 · −0,17 · −0,15	0,71 · 1,24 · 0,34 · 0,73	A	1	0,4	−0,405

```
particiones con λ ≤ 0:  2   ({1,2} y {3,4})
PBO = 2 / 6 = 0,33
```

Fíjate en que la elegida no es siempre la misma: B tres veces, A tres veces. En los datos reales pasa igual. En G se eligen 51 variantes distintas a lo largo de las 12.870 particiones, y la más repetida (`P00456`) sale 3.268 veces, una de cada cuatro. No existe «la variante elegida»: existe un procedimiento que elige.

2.9 La probabilidad de perder

El PBO compara con las demás variantes. Otra pregunta, más directa: **la elegida, ¿ganó o perdió dinero fuera?**

```
«pierde» = (nº de particiones en las que la suma de las celdas de la elegida, fuera, es
negativa) / (nº de particiones)
```

(`verdict/summary.py:56` ; la suma viene de `measure/cscv.py:194` .) En el ejemplo, la elegida suma fuera −1, +1, +2, +1, +5, +4: pierde en una de seis, **17 %**.

Las dos lecturas no dicen lo mismo, y por eso el estudio escribe las dos:

- **F `15.9.352` :** PBO 0,350 (`pass`), pero su elegida **pierde en el 55 %** de las particiones. Queda por encima de la mediana porque la mediana pierde más.
- **E `3.11.426` :** PBO 0,653 (`fail`), y su elegida **pierde sólo en el 7 %**. Queda por debajo de la mediana, pero casi todas las variantes ganan.

«Pierde» depende del nivel general de la estrategia, no de si elegir sirve. No sustituye al PBO ni el PBO a ella.

2.10 La degradación: dos pendientes distintas con el mismo nombre

«Degradación del rendimiento» responde a: lo que se ve dentro, ¿cuánto se conserva fuera? Se mide con una **recta de regresión**: se dibuja una nube de puntos con el Sharpe de dentro en horizontal y el de fuera en vertical, y se ajusta la recta que mejor la atraviesa. La **pendiente** dice cuánto sube «fuera» por cada punto que sube «dentro». Pendiente +1: lo de dentro se repite fuera. Pendiente 0: lo de dentro no dice nada. Pendiente negativa: cuanto mejor dentro, peor fuera. **R^2** (entre 0 y 1) dice qué parte de la dispersión explica la recta.

Hay dos nubes posibles, y el estudio usa las dos en sitios distintos:

(a) La del proyecto — «Pendiente» en `cscv.json`. En **cada partición**, un punto por **variante** (N puntos): su Sharpe dentro contra su Sharpe fuera. Se ajusta la recta, y luego se hace la **media de las 12.870 pendientes** (`measure/cscv.py:135-143`, `verdict/summary.py:40-41`). Pregunta: ¿las variantes buenas dentro son las buenas fuera? Con el ejemplo, partición {1,2}:

```
dentro:  0,71   1,24   0,34   0,73    media 0,755
fuera:   1,16  −0,20  −0,17  −0,15    media 0,160
la recta que mejor ajusta esos cuatro puntos tiene pendiente −0,19
```

Las seis pendientes: −0,19 · +0,39 · +1,72 · +0,17 · +0,20 · −0,06. Media: **+0,37**.

(b) La del artículo — sólo se dibuja. Un punto por **partición** (12.870 puntos): el Sharpe dentro **de la elegida** contra el Sharpe fuera **de la elegida**. Una sola recta (`contract.py:155-165`, `np.polyfit`). Pregunta: cuando la elegida sale muy alta dentro, ¿sale más baja fuera? En el ejemplo la pendiente es **−0,10**.

El balancín. La (b) sale negativa casi siempre, y no por sobreajuste. El motivo es aritmético: el histórico entero es fijo. Si una variante tiene dentro una media más alta que la suya de todo el histórico, fuera **tiene** que tenerla más baja, porque dentro y fuera son las dos mitades del mismo total. Míralo en la tabla: A da 2,35 dentro de {2,3} y 0,50 fuera; en la partición espejo, al revés. La ficha `knowhow/research/cscv-chosen-point-slope-seesaw.md` lo tiene medido: −0,57 sobre ruido puro. En las diez madres reales la pendiente (b) va de −0,50 a −0,92. No distingue una de otra y **no se guarda** en `cscv.json`.

La (a) no tiene ese problema en media (sobre los paneles reales sin diferencias entre variantes sale 0, apartado 2.12), pero tampoco es limpia: en el caso extremo en que todas las variantes tuvieran **exactamente** el mismo Sharpe en el histórico entero, el balancín haría que valiera −1 y que el PBO valiera 1. Lo he comprobado igualando las columnas a mano. En datos reales no se da.

2.11 La dominancia estocástica

Una forma de decir «la elegida es mejor que una variante corriente» sin resumirlo en un solo número. Se comparan dos listas de 12.870 valores: el Sharpe fuera **de la elegida** en cada partición y el Sharpe fuera **de la variante mediana** en cada partición.

- **Dominancia de primer orden:** ordenas las dos listas de menor a mayor. La elegida domina si su peor valor es mejor que el peor de la mediana, su segundo peor mejor que el segundo peor, y así **en todos los puestos**. En cualquier tramo de la distribución sale ganando.

- **Dominancia de segundo orden:** más floja. Se le permite perder en algún puesto si lo que gana acumulado hasta ahí lo compensa. Mirando de abajo arriba, nunca va por detrás en el total acumulado.

Con el ejemplo:

```
Sharpe fuera de la elegida, ordenado:   −0,20   0,17   0,50   0,71   0,83   0,87
Sharpe fuera de la mediana, ordenado:   −0,16   0,16   0,37   0,63   0,72   0,78
```

Gana en cinco puestos de seis. Pero en el primero, −0,20 es peor que −0,16: **no hay dominancia de primer orden**. Y como el fallo está en el puesto más bajo, no hay nada acumulado antes que lo compense: **tampoco de segundo**. Resultado: «ninguna».

El código lo hace sobre una rejilla de 200 puntos entre el mínimo y el máximo (`verdict/summary.py:74-83`): calcula para cada punto qué fracción de cada lista queda por debajo y exige que la de la elegida nunca supere a la de la mediana.

La trampa. Como el fallo que más pesa es el del extremo inferior, basta que la peor partición de la elegida sea peor que la peor partición de la mediana. La mediana de 300 variantes es, por construcción, un valor templado; la elegida es una sola variante y tiene extremos. Así que la prueba suspende casi siempre (apartado 3.6).

2.12 Qué PBO da el azar: la distribución nula

Esta es la cuenta que decide cómo se lee todo lo demás. El artículo dice «0,5 = elegir no sirve». Pero un PBO medido es **un** número sacado de **un** histórico. ¿Cuánto baila?

Por qué no basta contar particiones. 12.870 parece una muestra enorme. No lo es: todas las particiones salen de los mismos 16 bloques, y dos particiones que comparten siete de ocho bloques dan casi lo mismo. La información es la de 16 bloques, no la de 12.870 ensayos.

Lo que he hecho (medido por mí; no está en el estudio). Para C, E, F y G:

1. Tomo el panel real de la madre.
2. Igualo todas las columnas al **mismo Sharpe** (el mediano del panel): a cada una le resto su media, la divido por su desviación y le sumo la misma constante (`Z = (X − media)/sd + c`). Ahora ninguna variante es mejor que otra de verdad, pero conservan su parecido entre ellas y sus días raros. Efecto lateral: los días sin posición dejan de valer cero. No afecta al resultado.
3. Sorteo los días con reposición (los mismos días para todas las columnas) para fabricar un histórico alternativo.
4. Paso el CSCV con 16 bloques y la regla Argmax, y apunto el PBO.
5. Repito 150 veces por madre.

madre	media	sd	percentil 5	mediana	percentil 95
C	0,498	0,192	0,185	0,475	0,812
E	0,504	0,182	0,197	0,511	0,781
F	0,499	0,192	0,185	0,500	0,789
G	0,494	0,184	0,175	0,510	0,776

Juntando los 600 sorteos: **sd 0,187**; percentil 5 = 0,184 ± 0,02 (a dos errores típicos); tramo central del 95 %, de 0,15 a 0,82. El README del estudio da 0,21 con doce paneles sintéticos de 252 particiones (`verdict/README.md:9`); cuadra. El revisor (Taleb), con un panel gaussiano propio, obtiene sd 0,14; la diferencia cuadra con las colas gordas de estas celdas (curtosis de 18 a 206), sin aislar. **La sd es de cada panel: entre 0,14 y 0,21.** El centro en 0,5 no depende de nada de eso. Tampoco depende del nivel común de Sharpe: F está igualada a un Sharpe anual de −0,12 y da la misma campana que G, igualada a +0,46; el revisor obtiene 0,51 con nivel 0, +1 y −0,5.

Qué fracción de esos 600 PBO «sin nada que elegir» cae a cada lado de cada umbral que se ha propuesto:

umbral	fracción de nulos que lo cumple
PBO ≤ 0,08	0,7 % ± 0,7
PBO ≤ 0,15	2,5 % ± 1,3
PBO ≤ 0,18	4,8 % ± 1,8
PBO ≤ 0,25	11 %
PBO ≤ 0,30	16,2 % ± 3,0
PBO > 0,50 (el `fail` cargado)	**50,7 % ± 4,1** (46-56 % según la madre)
PBO > 0,60	32,0 % ± 3,8
PBO ≥ 0,71	15,5 % ± 3,0
PBO > 0,80	4,2 % ± 1,6

(Contados sobre los 600 valores; «±» es dos errores típicos de la proporción.)

Y dónde caen las cuatro madres reales dentro de su propia nula:

madre	PBO real	nulos con PBO igual o menor	pendiente real	nulos con pendiente igual o mayor
G	0,073	0 de 150	+0,562	1,3 %
C	0,292	15 %	+0,596	2 %
F	0,350	23 %	+0,126	25 %
E	0,653	77 %	−0,094	71 %

Lo que se saca:

- **G es la única de las cuatro por debajo de todos sus nulos.** «0 de 150» sólo acota p < 0,02, y sin corregir por haber mirado diez madres. `3.9.284` (0,140) queda bajo el percentil 5 conjunto sin nula propia: es un indicio. Que dos o más de diez caigan bajo el percentil 5 por azar ocurre el 8,6 % de las veces si las madres fueran independientes, y más si comparten histórico, como aquí.

- **Falta la potencia.** No rechazar la nula no es confirmarla. El revisor simuló variantes con diferencias verdaderas: con una dispersión de Sharpe de 0,05 anual (correlación 0,9) el PBO medio es 0,32 y sólo el 8 % de las veces baja de 0,18. Los seis `pass` entre 0,29 y 0,45 son compatibles con la nula **y** con un orden real modesto. La curva de potencia sobre los paneles reales no está hecha.
- **C tiene un PBO corriente (0,292, dentro del 15 % inferior) y una pendiente rara (+0,596, sólo el 2 % de los nulos llega).** Las dos medidas no dan la misma respuesta. La pendiente usa todas las variantes y el PBO sólo la elegida; la pendiente es menos ruidosa.
- **E está en mitad de la campana.** Su `fail` no dice que elegir sea contraproducente. Dice que no se ve que elegir sirva. Para afirmar lo primero haría falta un PBO por encima de 0,8.
- **La pendiente del proyecto tiene media 0 bajo la nula**, con sd de 0,15 (E) a 0,28 (C y G). Su intervalo del 5 al 95 %: de −0,4 a +0,5 en C y G; de −0,25 a +0,3 en E y F.

Límites de esta cuenta. Sortear días sueltos rompe la dependencia de un día con el siguiente; según la simulación del revisor eso no cambia la nula (con dependencia de ±0,4 entre días seguidos, sd 0,137 y 0,138). Son 150 sorteos y cuatro madres. «Todas las variantes iguales» cubre también «la estrategia no tiene ventaja», porque el PBO no depende del nivel común (párrafo anterior). Un matiz mío, no medido: una madre sin ventaja cuyas variantes operan con frecuencias distintas paga costes distintos, y eso es un orden verdadero entre variantes; podría dar PBO bajo sin que haya nada que ganar. Lo que falta de verdad es la curva de potencia.

La consecuencia incómoda. El PBO es una medida de **rangos**: ve el orden, no el tamaño de las diferencias. Un PBO bajo certifica **un orden estable entre variantes, aunque las diferencias sean despreciables; no dice si hay meseta ni lo niega**. Según la simulación del revisor, una dispersión verdadera de Sharpe de 0,10 anual entre variantes (con correlación 0,9) ya da PBO medio 0,09; con correlación 0,99 bastan 0,05 (PBO 0,008). Eso, en dinero, es una meseta. Y al revés: variantes exactamente iguales dan 0,5. Así que el PBO no sirve para decir «es meseta» ni «no lo es»; para eso hay que mirar el tamaño de las diferencias (el SPP, la nube de parámetros), no su orden.

2.13 El Sharpe deflactado (DSR)

Segunda pieza del estudio, independiente del PBO. Pregunta: el Sharpe de la mejor variante, ¿es más alto de lo que saldría por haber probado tantas?

Paso 1: cuánto Sharpe da el azar al probar N cosas. Si pruebas N variantes sin ventaja y sus Sharpes se reparten con desviación σ, el mayor de los N no es cero: es, de media,

```
listón = σ × [ (1 − γ) × Z(1 − 1/N)  +  γ × Z(1 − 1/(N × e)) ]
```

donde γ = 0,5772 (constante de Euler), e = 2,718, y Z(p) es el valor de una campana normal que deja por debajo una fracción p (Z(0,75) = 0,6745; Z(0,975) = 1,96). Es `expected_max_sharpe` en `core/surface/plateau.py`. Cuantas más pruebas, más alto el listón.

Paso 2: cuántas pruebas son de verdad. 317 variantes de la misma madre no son 317 pruebas independientes: comparten casi todas las operaciones. El estudio las **agrupa** por parecido (`core/surface/trials.py`): mide la correlación ρ entre cada par de curvas, la convierte en distancia √((1 − ρ)/2), junta las cercanas y prueba con 2, 3, … hasta 40 grupos (`cluster_k_max: 40`). Se queda con el

número de grupos que da mejor **silueta**. La silueta, de −1 a 1, mide lo bien separados que quedan los grupos: cerca de 1, grupos nítidos; por debajo de 0,2, el propio código avisa de que no hay estructura (es una nota de lectura; no se aplica). El número de grupos es `n_clusters`, y es el N del paso 1.

Paso 3: la probabilidad. Con el Sharpe observado SR, el listón, el número de días n, la asimetría (*skew*: si la cola larga está a la derecha o a la izquierda) y la curtosis (cuánto pesan los días extremos; una campana normal tiene 3):

```
z   = (SR − listón) × √(n − 1)  /  √( 1 − asimetría × SR + (curtosis − 1)/4 × SR² )
DSR = fracción de una campana normal que queda por debajo de z
```

DSR = 0,93 se lee: «con estos datos, hay un 93 % de probabilidad de que el Sharpe verdadero de la mejor variante supere al que daría el azar entre `n_clusters` pruebas».

La cuenta con C `14.3.479`, a mano:

```
mejor variante dentro (build + oos1):  P00096
SR diario = 0,04521 (0,72 anual)   n = 3.014 días   asimetría = 6,098   curtosis = 80,07
σ de los Sharpes de las 317 variantes = 0,0201      n_clusters = 4

Z(1 − 1/4) = Z(0,75) = 0,6745        Z(1 − 1/(4 × 2,718)) = Z(0,908) = 1,3287
listón = 0,0201 × (0,4228 × 0,6745 + 0,5772 × 1,3287) = 0,0201 × 1,0521 = 0,02115

numerador   = (0,04521 − 0,02115) × √3.013 = 0,02406 × 54,89 = 1,3207
denominador = √(1 − 6,098 × 0,04521 + (80,07 − 1)/4 × 0,04521²) = √0,7647 = 0,8745
z   = 1,3207 / 0,8745 = 1,510
DSR = 0,9345
```

El `cscv.json` de C guarda `dsr` 0,9345 y `sharpe_benchmark` 0,0211. ✓

Lo que hay que saber del DSR tal como está:

- **Depende muchísimo de `n_clusters`.** Con N = 4 grupos, C saca 0,93. Con N = 317 (cada variante una prueba), saca **0,20**. La verdad está en medio y nadie sabe dónde; la silueta de C es 0,47, de las claras.
- **Sólo descuenta las variantes de esta madre.** No descuenta que la madre salió de un generador que probó miles de estrategias, ni los pasos 8 a 16. Es un listón bajo.
- **Se calcula con el `build` dentro** (apartado 5.10).
- **Curtosis de 80** quiere decir que unos pocos días concentran casi todo el movimiento. Con celdas que son «mínimo de hoy menos mínimo de ayer» y más de la mitad a cero, es lo esperable; es otra razón para no leer este número con decimales.

2.14 El bootstrap y el intervalo

Para poner un margen a un número se usa el **bootstrap**: se fabrica una muestra alternativa sorteando con reposición de la que tienes, se recalcula el número, se repite muchas veces, y el intervalo del 95 % es el tramo que deja fuera el 2,5 % más bajo y el 2,5 % más alto de las repeticiones.

El estudio lo usa en un solo sitio: el percentil OOS de la partición cronológica (apartado 3.8), con 200 repeticiones (`bootstrap: 200`) y sorteando **variantes**, no días. Pregunta que contesta: si la fábrica hubiera producido otro lote de variantes, ¿dónde habría caído la elegida?

El PBO no lleva intervalo. El estudio no le calcula ninguno. Yo probé el bootstrap directo de días sobre el panel real y sale sesgado hacia abajo (en C, media 0,16 frente al 0,292 real), porque trata las diferencias de la muestra como si fueran verdaderas. No lo doy. La referencia válida es la nula del apartado 2.12.

2.15 Lo que el CSCV no puede ver

- **Todas las particiones viven en el mismo histórico.** Si el histórico entero favoreció a un tipo de parámetro, las 12.870 particiones lo confirman.
- **Las variantes son hijas de una madre ya elegida.** El CSCV mide el sobreajuste de elegir **entre ellas**, no el de haber llegado a esa madre.
- **No dice nada del nivel.** Una estrategia que pierde con todas sus variantes puede sacar PBO 0,05 si pierde con orden.
- **Usa `oos2`.** Después de este paso, `oos2` ya no es un tramo que nadie haya mirado, en el sentido de que su orden entre variantes ha entrado en un número que decide.

---

3. Lo que calcula el estudio, pieza a pieza

Todo acaba en `<lote>/cscv.json` (`report.py:174`) y en `<lote>/estudios/cscv.*` para la ventana (`report.py:176`). El JSON tiene los números sueltos en el primer nivel, con el nombre de la regla como sufijo (`pbo_argmax`, `pbo_plateau_centre`, …), y cuatro objetos: `rules`, `trials`, `sharpe` y `drift`.

3.1 El tamaño del panel: `n`, `periods`, `period`, `blocks`, `partitions`, `score`

campo	qué es	C `14.3.479`
`n`	variantes que entran, tras el suelo de 50 operaciones por lado	317
`periods`	días	3.722
`period`	`D`, diario	D
`blocks`	S	16
`partitions`	C(16,8)	12.870
`score`	con qué se puntúa	sharpe

`n` es el número que más conviene mirar antes de leer nada más: es el conjunto sobre el que se elige. La tabla del apartado 1.3 enseña que no es «las variantes fabricadas»: es lo que queda después de dos suelos.

3.2 `pbo_argmax` — el número del paso

Fórmula: apartado 2.7. Código: `verdict/summary.py:21`. Las diez, con lo que rodea al número:

madre	N	PBO	ω mediano	elegida sobre la mediana	variantes distintas elegidas	la más elegida
`10.11.484`	321	**0,385**	0,618	61,5 %	84	`P00465`, 3.377 veces
`13.3.390`	215	**0,642**	0,333	35,8 %	92	`P00450`, 2.726
C `14.3.479`	317	**0,292**	0,755	70,8 %	67	`P00447`, 3.848
F `15.9.352`	430	**0,350**	0,626	65,0 %	121	`P00452`, 3.357
`16.5.352`	438	**0,453**	0,542	54,7 %	118	`P00286`, 2.126
`18.5.355`	265	**0,346**	0,594	65,4 %	99	`P00026`, 2.406
`19.2.363`	352	**0,391**	0,589	60,9 %	78	`P00001`, 2.567
E `3.11.426`	273	**0,653**	0,332	34,7 %	123	`P00228`, 2.497
`3.9.284`	327	**0,140**	0,777	86,0 %	70	`P00075`, 2.559
G `4.13.338`	417	**0,073**	0,900	92,7 %	51	`P00456`, 3.268

Los diez PBO coinciden con los guardados. ✓

Cómo se reparte λ en tres de ellas (mínimo · p5 · p25 · mediana · p75 · p95 · máximo):

```
G:  −5,34   −0,23   +0,78   +2,19   +3,29   +4,41   +6,03    casi todo a la derecha del cero
C:  −5,76   −0,95   −0,15   +1,12   +2,70   +4,65   +5,76    tres cuartos a la derecha
E:  −5,61   −3,38   −1,89   −0,70   +0,42   +1,50   +3,50    dos tercios a la izquierda
```

Incluso en G hay particiones en las que la elegida queda la última (λ = −5,34 es ω ≈ 0,005).

Cuánto cambia el PBO con el número de bloques. Recalculado por mí con S = 8, 10, 12, 14 y 16:

madre	S = 8	10	12	14	16
`10.11.484`	0,257	0,413	0,275	0,297	**0,385**
`13.3.390`	0,629	0,599	0,687	0,629	**0,642**
C	0,271	0,298	0,197	0,228	**0,292**
F	0,343	0,365	0,368	0,378	**0,350**
`16.5.352`	0,429	0,476	0,418	0,443	**0,453**
`18.5.355`	0,443	0,365	0,460	0,281	**0,346**
`19.2.363`	0,243	0,349	0,299	0,382	**0,391**
E	0,643	0,679	0,574	0,650	**0,653**
`3.9.284`	0,186	0,159	0,092	0,096	**0,140**
G	0,071	0,107	0,110	0,078	**0,073**

Dentro de una misma madre el PBO se mueve hasta 0,18 según S (`18.5.355`: de 0,281 a 0,460). Pero **el lado del 0,5 no cambia en ninguna**: las dos que suspenden suspenden con los cinco valores de S, y las ocho que pasan, pasan. Y G y `3.9.284` quedan por debajo de 0,19 con todos. La elección de 16 no decide el veredicto aquí.

3.3 `median_omega_argmax`

El puesto típico de la elegida, fuera. Mediana de los 12.870 ω (`summary.py:95-97`). G: 0,900 (la elegida queda típicamente por delante del 90 %). E: 0,332. Dice lo mismo que el PBO con más matiz: un PBO de 0,39 con ω mediano 0,59 (`19.2.363`) es «un poco por encima del medio», no «bien».

3.4 `prob_loss_argmax` — «pierde»

Fórmula: apartado 2.9. Código: `summary.py:56`.

madre	PBO	pierde	Sharpe anual medio de la elegida: dentro → fuera
`10.11.484`	0,385	**3,1 %**	0,97 → 0,46
`13.3.390`	0,642	**26,2 %**	0,69 → 0,15
C	0,292	**10,2 %**	0,77 → 0,38
F	0,350	**54,5 %**	0,47 → −0,05
`16.5.352`	0,453	**33,8 %**	0,77 → 0,11
`18.5.355`	0,346	**4,2 %**	0,93 → 0,40
`19.2.363`	0,391	**51,3 %**	0,66 → 0,00
E	0,653	**7,0 %**	1,00 → 0,37
`3.9.284`	0,140	**2,1 %**	1,06 → 0,68
G	0,073	**0,2 %**	1,09 → 0,77

La última columna la he calculado yo (el estudio no la guarda): la elegida pierde de media entre un tercio (G) y todo (F, `19.2.363`) de su Sharpe al pasar de la mitad en que se eligió a la otra. Es la medida más directa de cuánto infla elegir, y no depende de comparar con nadie.

Dos madres con `pass` tienen una elegida que pierde dinero más de la mitad de las veces (F y `19.2.363`). El veredicto no lo mira; va sólo en la nota de la ventana («pierde 54,5 %»).

3.5 `slope`, `r2`, `slope_sd` — la pendiente del proyecto

Fórmula: apartado 2.10 (a). Código: `measure/cscv.py:135-143` y `summary.py:40-41`. `slope` es la media de las 12.870 pendientes; `r2`, la media de los R^2; `slope_sd`, la desviación típica de las pendientes entre particiones. En el primer nivel del JSON, `slope` y `r2` son los de la primera regla; las copias por regla son idénticas, porque la pendiente usa todas las variantes y no depende de cuál se elige.

madre	pendiente	sd entre particiones	pendiente del artículo (sólo dibujada)
`10.11.484`	**+0,302**	0,185	−0,63
`13.3.390`	**−0,157**	0,136	−0,92
C	**+0,596**	0,306	−0,85
F	**+0,126**	0,197	−0,68
`16.5.352`	**+0,063**	0,107	−0,76
`18.5.355`	**+0,428**	0,146	−0,50
`19.2.363`	**+0,228**	0,160	−0,74
E	**−0,094**	0,113	−0,87
`3.9.284`	**+0,744**	0,256	−0,70
G	**+0,562**	0,262	−0,92

Cómo leer `slope_sd`. No es el error de la pendiente. Las 12.870 pendientes no son independientes (apartado 2.12), así que dividir por √12.870 no vale. Para saber si una pendiente es distinta de cero hay que compararla con su nula: sd de 0,15 a 0,28 según la madre. Con eso, +0,60 (C), +0,74 (`3.9.284`) y +0,56 (G) están claramente fuera; +0,13 (F) y +0,06 (`16.5.352`) no se distinguen de cero; −0,16 y −0,09 tampoco.

La pendiente del artículo es negativa en las diez, G incluida. Por el balancín (2.10) no distingue la mejor de la peor. Está bien que no se guarde; conviene saber que la gráfica «decay» de la ventana enseña ésa y no la del JSON.

3.6 `dominance_argmax`

Fórmula: apartado 2.11. Código: `summary.py:74-83`. Valores posibles: `primer orden`, `segundo orden`, `ninguna`.

En la corrida: `ninguna` en las diez, con las dos reglas. En las diez, el peor Sharpe fuera de la elegida es peor que el peor de la variante mediana. El caso más claro es el de la mejor, G:

```
G, Sharpe anual fuera, a lo largo de las 12.870 particiones:
                    mínimo     media
elegida             −0,39      0,77
variante mediana    −0,12      0,47
```

La elegida gana a la mediana en el 92,7 % de las particiones y de media saca 0,30 más. Pero su peor partición (−0,39) es peor que la peor de la mediana (−0,12), y con eso suspende los dos órdenes. La ventana lo resume como «no gana».

Un dato que es igual para las diez no separa nada. Además, la descripción de la función dice que la comparación es «emparejada» (partición a partición) y el código compara las dos listas ordenadas cada una por su lado; para la dominancia es lo correcto, pero el texto no describe lo que se hace.

3.7 La segunda regla: `…_plateau_centre`

En vez de elegir el máximo dentro, se elige **la variante cuyo vecindario tiene mejor media**. Cómo:

1. A cada parámetro se le numeran sus valores distintos, de menor a mayor: nivel 0, 1, 2… (`measure/rules.py:31`).
2. Dos variantes son **vecinas** si en **todos** los parámetros a la vez están a `plateau_radius` niveles o menos (`rules.py:87`; `plateau_radius: 1`).
3. La puntuación de cada variante es la media del Sharpe de ella y sus vecinas, y se elige el máximo de esa media (`rules.py:68-69`).

La idea es la tuya: no quedarse con un pico, sino con el centro de una zona buena. El problema está en el paso 1: los niveles se numeran sobre **todos los valores que aparecen**, y el estrato de cobertura mete valores que no caen en la rejilla. Un parámetro llega a tener 85 niveles. A radio 1, y exigiendo cercanía en todos los parámetros a la vez, casi nadie tiene vecinos:

madre	vecinos por variante (mínimo · mediana · máximo; se cuenta a sí misma)	variantes sin ningún vecino	las dos reglas eligen lo mismo	PBO Argmax	PBO plateau
`10.11.484`	1 · 1 · 77	73 %	94,5 %	0,385	0,385
`13.3.390`	1 · 1 · 1	**100 %**	100 %	0,642	0,642
C	1 · 1 · 84	69 %	85,8 %	0,292	0,297
F	1 · 2 · 51	49 %	70,7 %	0,350	0,338
`16.5.352`	1 · 3 · 43	21 %	24,9 %	0,453	**0,598**
`18.5.355`	1 · 2 · 50	32 %	17,8 %	0,346	**0,630**
`19.2.363`	1 · 2 · 3	38 %	52,8 %	0,391	0,459
E	1 · 2 · 13	44 %	75,3 %	0,653	0,651
`3.9.284`	1 · 1 · 87	70 %	23,4 %	0,140	**0,317**
G	1 · 1 · 81	77 %	84,8 %	0,073	0,084

Para una variante sin vecinos, «la media de su vecindario» es su propio Sharpe: la regla del plateau es Argmax. En `13.3.390` (cuyo lote no tiene estrato de vecindario) son la misma regla.

Donde sí difieren, el plateau **no mejora**: igual o peor en 8 de 10. Pero las diferencias están dentro del ruido (sd nula 0,14-0,21) salvo en tres: `16.5.352`, `18.5.355` y `3.9.284`, que suben más de 0,14. El README del estudio dice que el plateau bajó el PBO «del 30 % al 4 %» en otro lote; aquí no se ve. Por qué empeora no lo he medido. Una explicación posible, sin probar: las pocas variantes con muchos vecinos (hasta 87) son las del estrato de vecindario, pegadas a la madre; promediar 80 Sharpes da una puntuación muy estable, de modo que la regla acaba eligiendo casi siempre «la zona de la madre» frente a variantes sueltas que compiten con su Sharpe en bruto.

Si la regla del plateau fuera la que decide, suspenderían **4 de 10** (`13.3.390`, `16.5.352`, `18.5.355`, E) en vez de 2.

3.8 `pct_oos_<regla>` y `ci95` — el coste de la regla en la partición de verdad

Deja el CSCV y mira la única partición cronológica: se elige con `build` + `oos1` y se comprueba en `oos2`.

> `pct_oos = % de variantes cuyo Sharpe en oos2 es menor que el de la elegida`

(`verdict/cost.py:30-32`.) Es un ω en tanto por ciento y sin el «+1». El intervalo `ci95` sale de sortear **variantes** 200 veces y repetir la elección (`cost.py:53-61`, `core.surface.dedupe.bootstrap_ci`). `ci95` sólo está dentro del objeto `rules`, no en el primer nivel.

madre	elegida	percentil OOS Argmax	IC 95 %	percentil OOS plateau	IC 95 %
`10.11.484`	`P00465`	63,9	5,3 – 92,9	63,9	4,7 – 93,5
`13.3.390`	`P00001`	5,1	2,8 – 87,5	5,1	2,8 – 87,5
C	`P00096`	89,0	66,9 – 91,8	67,8	64,7 – 92,4
F	`P00452`	50,0	5,1 – 75,6	50,0	16,5 – 74,7
`16.5.352`	`P00007`	13,5	10,7 – 86,3	5,7	1,8 – 85,9
`18.5.355`	`P00026`	34,7	29,4 – 56,6	74,0	29,0 – 90,3
`19.2.363`	`P00001`	49,7	45,2 – 70,9	49,7	45,2 – 67,2
E	`P00214`	75,8	16,1 – 80,3	63,7	53,8 – 89,4
`3.9.284`	`P00064`	80,1	74,9 – 98,8	2,4	0,9 – 92,8
G	`P00456`	94,2	59,7 – 95,7	94,2	85,9 – 95,9

La cuenta de F: 215 de 430 variantes tienen en `oos2` un Sharpe menor que `P00452` → 215 / 430 = 50,0 %. ✓

Lo que se ve:

- **Los intervalos son enormes.** Cinco de diez cubren más de 60 puntos. Con sólo cambiar qué variantes hay en el lote, la elegida pasa de la cola baja a la alta. Una partición cronológica no da para afirmar nada de la regla; por eso existe el CSCV.
- **El número no cuenta la misma historia que el PBO.** E, que suspende el PBO, cae en el percentil 76 en `oos2`. `16.5.352`, que lo aprueba, cae en el 13.
- **En dos madres la elegida con `build` + `oos1` es `P00001`, un canario**: la permutación del SPP con mayor beneficio en IS, metida como control (apartado 5.7).

3.9 `drift` — cuánto se mueve el mejor punto

Se busca la mejor variante dentro (`is_best`) y la mejor en `oos2` (`oos_best`), y se cuenta a cuántos **niveles** de distancia están (`cost.py:80-85`): `levels_max` es la distancia en el parámetro que más se mueve; `levels_total`, la suma en todos.

madre	`levels_max`	`levels_total`
`10.11.484`	27	65
`13.3.390`	40	91
C	54	193
F	46	102
`16.5.352`	11	25
`18.5.355`	8	21
`19.2.363`	37	59
E	75	116
`3.9.284`	2	4
G	44	131

`levels_max` también está en el primer nivel del JSON.

Cuidado con la unidad. Un «nivel» no es un paso de rejilla: es un puesto en la lista de valores distintos que aparecen, cobertura incluida (apartado 3.7). Con hasta 85 niveles por parámetro, 44 niveles de distancia pueden ser medio rango. Y entre dos madres no se comparan, porque cada una tiene su número de niveles. G, la del mejor PBO, se mueve 44; `16.5.352`, con PBO 0,45, se mueve 11. Hoy este número no ordena nada.

3.10 `trials` — cuántas pruebas valen las variantes

Apartado 2.13, paso 2. Código: `core/surface/trials.py`, llamado desde `verdict/trials.py`. Se calcula sobre la ventana de dentro (`build` + `oos1`).

madre	N	`n_clusters`	silueta	correlación mediana entre variantes
`10.11.484`	321	6	0,38	0,39
`13.3.390`	215	3	0,23	0,54
C	317	4	0,47	0,52
F	430	18	0,25	0,39
`16.5.352`	438	5	0,13	0,27
`18.5.355`	265	6	0,14	0,40
`19.2.363`	352	5	0,15	0,36
E	273	36	0,15	0,45
`3.9.284`	327	8	0,25	0,41
G	417	35	0,47	0,46

La última columna la he calculado yo. En **cuatro de diez** la silueta está por debajo de 0,2: el propio código dice que ahí no hay grupos y que el número elegido (5, 6, 5, 36) es poco más que arbitrario. El algoritmo además rechaza los repartos en que un grupo se queda con más de la mitad de las variantes, lo que empuja hacia más grupos.

Una correlación mediana de 0,4-0,5 entre curvas diarias de variantes de **la misma estrategia** es más baja de lo que cabría esperar. Encaja con que las celdas sean incrementos del mínimo diario y no ganancias (apartado 5.1): dos variantes con las mismas operaciones pero distinto tamaño de stop tienen mínimos diarios distintos.

3.11 `sharpe` y `dsr` — el Sharpe deflactado

Apartado 2.13. Código: `verdict/trials.py:31-34`, `core/surface/plateau.py`. Siempre sobre Sharpe (aunque el `score` fuera Sortino) y siempre sobre la ventana de dentro.

madre	mejor dentro	Sharpe anual	asimetría	curtosis	listón (anual)	DSR	DSR si cada variante contara como una prueba
`10.11.484`	`P00465`	0,94	2,0	32	0,27	**0,992**	0,880
`13.3.390`	`P00001`	0,57	6,7	206	0,11	**0,959**	0,789
C	`P00096`	0,72	6,1	80	0,34	**0,935**	0,202
F	`P00452`	0,43	1,0	21	0,40	**0,546**	0,225
`16.5.352`	`P00007`	0,69	0,0	18	0,25	**0,935**	0,574
`18.5.355`	`P00026`	1,05	1,1	19	0,36	**0,993**	0,823
`19.2.363`	`P00001`	0,65	3,9	69	0,24	**0,938**	0,595
E	`P00214`	0,76	3,6	99	0,36	**0,932**	0,853
`3.9.284`	`P00064`	0,92	3,3	61	0,44	**0,963**	0,556
G	`P00456`	1,04	1,8	37	0,48	**0,977**	0,901

Los diez DSR y los diez listones coinciden con los guardados. ✓ La última columna es mía.

Nueve de diez por encima de 0,93. No porque las nueve sean buenas, sino porque el listón es bajo: entre 3 y 36 «pruebas», y 3.014 días que hacen pequeño cualquier error. Sin deflactar nada (listón cero), las diez estarían entre 0,935 y 0,9999. El DSR sólo separa a F.

El DSR **no entra en el veredicto**. Aparece en la frase de la ventana.

3.12 El veredicto que ve la ventana

`contract.py:76-98`. Para G:

```
etiqueta:  «PBO 7,3 % con Argmax»
estado:    pass
score:     (vacío)
partes:    «Argmax: PBO 7,3 %»            pass   nota «pierde 0,2 % · no gana»
           «Centro del plateau: PBO 8,4 %» pass   nota «pierde 0,2 % · no gana»
```

Y una frase con el número de variantes, `n_clusters`, el DSR, la pendiente y el radio.

`score` y el `value` de las partes se dejan vacíos a propósito (`contract.py:81-82`): la ventana pintaría 0,07739 en crudo y un PBO se lee en porcentaje. Efecto lateral: el veredicto no lleva ningún número que una regla pueda leer.

El texto de ayuda del propio estudio dice «por debajo del 50 %… Pass; por encima… Fail» (`contract.py:34-38`) y el de la ventana «pass con PBO hasta el 50 %» (`ui/text/columnhelp_fields.py:12`). El código hace lo segundo: 0,5 exacto es `pass`.

3.13 Lo que se dibuja y no se guarda

- El histograma de λ.
- La nube «decay»: Sharpe anual de la elegida dentro contra fuera, con la recta del artículo (`contract.py:155-165`). Es la pendiente (b) del apartado 2.10, negativa en las diez.
- La tabla «Regla a regla» con PBO, percentil OOS, intervalo, «pierde» y dominancia.

3.14 Las filas del libro de miradas

Cada ejecución escribe una fila por tramo leído (tres) con el criterio `cscv/pbo` y los umbrales `blocks` y `min_trades` (`report.py:182-188`, `engines/variants/look.py`). Para estas madres las filas están en `AlgoData/ledger/NIKKEI225_M30_simple_long.jsonl` (180 filas: 60 lotes × 3 tramos). El fichero `NIKKEI225_M30_volatilidadL.jsonl` sólo tiene tres, de `14.3.479`. Por qué la familia apuntada es `simple_long` y no `volatilidadL` no lo he rastreado.

---

4. El criterio cargado y por qué

4.1 En `criteria.yaml`: nada

Las entradas que existen son `8`, `10`, `11.5`, `12`, `14` y `16`. No hay `"18"` ni `"20"`. El comentario de la línea 59:

> PROVISIONAL (Council 2026-10-04): do not freeze before a null population has run the chain. Steps 17, 18, 19, 21, 23, 24 annotate only: no rule on purpose.

Y no hay hechos que leer: `python3 -m pipeline.autopilot.facts --project Trade_NIKKEI225_M30_volatilidadL --step 18` devuelve una tabla vacía. Para el paso 20 devuelve nueve claves de `blindJoint.*` (el veredicto conjunto, el de la WFM, el del SPA y cuatro de referencia); ninguna es del CSCV.

4.2 El único umbral: PBO > 0,5

`contract.py:87`:

```
"fail" if pbo > 0.5 else "pass"
```

sobre la primera regla de `cscv.rules`, que es `argmax`.

De dónde sale el 0,5. De la definición del artículo: por encima de 0,5, lo más probable es que la elegida quede por debajo de la mediana, es decir, elegir es peor que no elegir. Es el punto de indiferencia teórico, no un umbral calibrado.

Lo que hace en la práctica (apartado 2.12): un lote sin diferencias entre variantes lo supera el 51 % de las veces (46-56 % según la madre). Como frontera de `fail` tiene una tasa de falsa alarma del 50 % contra la meseta perfecta; como frontera de `pass`, deja pasar al otro 50 %.

Hay una tercera copia del mismo umbral, en la cadena antigua de una sola madre: `pipeline/config.yaml:54-62`, puerta `cscv.pbo_argmax` con máximo 0,50, marcada PROPOSED. No es la que corre el autopiloto.

4.3 Los parámetros que sí están fijados

parámetro	valor	dónde	de quién
bloques S	16	`ledger/thresholds.yaml:69-75`	tuyo, 2026-09-30 (antes 12)
suelo de operaciones por lado	50	`engines/variants/config.yaml:19`	—
suelo de operaciones en total	100	`assets/_build.yaml:368`	—
reparto cronológico	`oos2_only`	`engines/variants/config.yaml:46`	—
periodo	diario	`cscv/config.yaml`	—
puntuación	Sharpe	`cscv/config.yaml`	—
reglas	Argmax, Centro del plateau	`cscv/config.yaml`	—
radio del plateau	1	`cscv/config.yaml`	—
repeticiones del bootstrap	200	`cscv/config.yaml`	—
máximo de grupos	40	`cscv/config.yaml`	—
semilla	20260922	`cscv/config.yaml`	—

S y el suelo de 50 van apuntados en cada fila del libro de miradas: cambiarlos es una mirada nueva.

4.4 Lo que se propuso y no se adoptó

Del informe anterior (`criterios-pasos-6-a-19-explicados-2026-10-06.md`, §9), con lo que dice la nula del apartado 2.12 de cada propuesta:

propuesta	de quién	fracción de nulos que la cumple	madres reales que la cumplen
PASA si PBO ≤ 0,15	Karpathy (retirada)	2,5 %	2 (G, `3.9.284`)
NO PASA si PBO ≥ 0,71	Karpathy (retirada)	15,5 %	0
sólo informa por debajo de 0,08	Kahneman (0,5 − 2 sd)	0,7 %	1 (G)
«entre 0,08 y 0,92 no se distingue del azar»	informe anterior	—	9 de 10 dentro

El informe anterior usaba sd 0,21 (la del README, sobre ruido sintético). Sobre los paneles reales sale 0,19: la misma lectura. La frase de Feynman que recogía —«no sé explicar qué certifica un PBO ≤ 0,15»— tiene ahora respuesta, y es incómoda: certifica que el orden entre las variantes es estable, aunque las diferencias sean despreciables; ni afirma ni niega una meseta (final del apartado 2.12).

4.5 Si pones el umbral en otro sitio

Sobre las diez madres. «Pasa» y «falla» son del PBO Argmax con la configuración cargada.

si pones…	fallan	quedan	fracción de nulos «meseta perfecta» que fallaría
`fail` si PBO > 0,50 (hoy)	2 (`13.3.390` , E)	8	50,7 % ± 4,1
`fail` si PBO > 0,60	2 (las mismas)	8	32,0 % ± 3,8
`fail` si PBO ≥ 0,71	0	10	15,5 % ± 3,0
`fail` si PBO > 0,80	0	10	4,2 % ± 1,6
exigir PBO ≤ 0,30	7	3 (C, `3.9.284` , G)	83,8 % ± 3,0
exigir PBO ≤ 0,18	8	2 (`3.9.284` , G)	95,2 % ± 1,8
exigir PBO ≤ 0,08	9	1 (G)	99,3 % ± 0,7

(Fracciones contadas sobre los 600 nulos; «±» es dos errores típicos.) Las tres últimas filas certifican «el orden entre variantes es estable». No certifican ni niegan una meseta: echarían a la estrategia cuyas variantes son exactamente iguales y dejarían pasar a otra con diferencias minúsculas pero ordenadas. Y tienen poca potencia: con un orden real modesto, la mayoría de las veces no se pasan (apartado 2.12).

Y con la misma frontera de 0,5 pero cambiando lo demás:

cambio	fallan
nada (cargado)	2: `13.3.390` 0,642 · E 0,653
reparto `oos1_oos2` en vez de `oos2_only`	2: `13.3.390` 0,653 · E 0,542
suelo de 30 en vez de 50	2: `13.3.390` 0,687 · E 0,561
sin suelo por lado	**1**: E 0,559 (`13.3.390` baja a 0,423)
regla del plateau como primera	**4**: `13.3.390` · `16.5.352` · `18.5.355` · E
S = 8, 10, 12 o 14	2, las mismas

---

5. Lo que no cuadra o no está medido

5.1 Las celdas son incrementos del mínimo diario, no ganancias del día

`knowhow/sqx-format/daily-equity-bin.md` (probado, 2026-10-04): `dailyEquity.bin` guarda el capital **más bajo** de cada día, muestreado mientras hay posición abierta; «no es una marca a mercado… las diferencias son de mínimo a mínimo, nunca P&L diario». El comentario de

`studies/optimisation/cscv/config.yaml` (hacia las líneas 11-13) dice lo contrario: que las curvas están marcadas a mercado y que el 75 % de las celdas se mueve. En esta corrida se mueve entre el 14 % y el 50 %.

Qué cambia: la media de cada columna casi no (los incrementos suman el total), la desviación sí, y con ella el Sharpe diario, la asimetría y la curtosis (de 18 a 206 aquí). La ficha da el efecto sobre el PBO en **poblaciones**: 0,37 → 0,35, 0,36 → 0,39, 0,40 → 0,52. Sobre un lote de variantes **no está medido**; reconstruir la ganancia diaria de verdad pide las operaciones de cada variante. Un salto de 0,40 a 0,52 cruza la frontera cargada.

Lo que **no** rompe (revisor): ningún supuesto del CSCV. Las filas siguen siendo del mismo día para todas las variantes y siguen sumando; la suma por bloque sólo yerra en los días frontera. Lo que cambia es el denominador del Sharpe y el peso de las colas, y eso ensancha la nula. Lo que sí invalida es leer el DSR con decimales.

5.2 «El PBO es el mismo en todas las composiciones» es falso

`engines/variants/config.yaml:40-42` y `engines/variants/panel.py:134-136` lo afirman, con la lógica de que el CSCV usa el histórico entero y el reparto sólo afecta al bloque cronológico. Pero `report.py:130` filtra el panel con el suelo de operaciones **del reparto elegido** antes de calcular nada. Medido:

madre	`oos2_only` , suelo 50 (cargado)	`oos1_oos2` , suelo 50	`oos2_only` , suelo 30	sin suelo
	N · PBO	N · PBO	N · PBO	N · PBO
`10.11.484`	321 · 0,385	391 · 0,352	356 · 0,383	420 · 0,343
`13.3.390`	215 · 0,642	232 · 0,653	228 · 0,687	276 · **0,423**
C	317 · 0,292	412 · **0,200**	375 · 0,238	417 · 0,199
F	430 · 0,350	501 · 0,406	478 · 0,387	501 · 0,406
`16.5.352`	438 · 0,453	499 · 0,489	474 · 0,473	499 · 0,489
`18.5.355`	265 · 0,346	362 · 0,448	332 · 0,436	376 · 0,462
`19.2.363`	352 · 0,391	374 · 0,400	402 · 0,418	402 · 0,418
E	273 · 0,653	412 · 0,542	425 · 0,561	429 · 0,559
`3.9.284`	327 · 0,140	416 · 0,156	385 · 0,159	416 · 0,156
G	417 · 0,073	423 · 0,075	460 · 0,076	460 · 0,076

El suelo no quita variantes al azar: quita las que operan poco en `oos2` , que son un tipo de variante. En E quita 156 de 429.

5.3 Los suelos no son los que dicen los comentarios

El comentario de `engines/variants/config.yaml:6` habla de un suelo duro de «50 operaciones en TOTAL» y el de la línea 12 dice que «30 es el suelo que usa el propio SQX». Los valores que corren son **100** en total (`assets/_build.yaml:368`) y **50** por lado (`config.yaml:19`).

5.4 El plateau a radio 1 es Argmax para la mayoría

Apartado 3.7. Entre el 21 % y el 100 % de las variantes están solas en su vecindario (`rules.py:31` numera como niveles los valores de cobertura; `rules.py:87` exige cercanía en todos los parámetros a la vez). El titular del README («del 30 % al 4 %») no se reproduce: aquí el plateau es igual o peor en 8 de 10. Que la causa sea la numeración de niveles lo deduzco del código y del recuento de niveles; no lo he probado cambiando la numeración. Y «peor» supera el ruido sólo en `16.5.352`, `18.5.355` y `3.9.284`.

5.5 La dominancia es «ninguna» por construcción

Apartado 3.6. Diez de diez, dos reglas. `summary.py:74-83`. El texto de la función dice «emparejada»; el código no empareja.

5.6 La «pendiente» no es la del artículo

Apartados 2.10 y 3.5. `slope` en `cscv.json` es la media, sobre particiones, de la pendiente entre **todas las variantes** (`summary.py:40`). La degradación del artículo (elegida dentro contra elegida fuera) se dibuja (`contract.py:157`) y no se guarda. Tienen signo contrario en 8 de 10. La frase de la ventana lo avisa («ajustada sobre todas las variantes, no solo la elegida; extensión de este proyecto»); el nombre del campo no.

5.7 Los controles y las copias compiten como candidatos

El panel incluye el origen, los canarios, las parejas inertes y columnas idénticas. Medido:

madre	particiones en que la elegida es un canario	PBO con → sin canarios	PBO con → sin columnas duplicadas
`19.2.363`	19,9 % (`P00001`, 2.567 veces)	0,391 → **0,441**	—
F	11,4 %	0,350 → 0,383	0,350 → 0,341
C	7,3 %	—	0,292 → 0,273
`13.3.390`	5,2 %	—	—
`3.9.284`	4,6 %	0,140 → 0,109	—
`10.11.484`	≤ 1,3 %	—	0,385 → 0,377
G	≤ 1,3 %	—	0,073 → **0,121**

(«—»: cambio de ±0,006 o menos, o sin duplicados.)

Tamaño de estos movimientos: 0,03-0,05, un cuarto de la sd nula. La aritmética es cierta (el conjunto cambia y el número cambia); que sea un efecto y no ruido no está demostrado.

`P00001` es la permutación del SPP con **mayor beneficio en IS**: un punto escogido por ser el mejor del `build`, metido en un panel cuya pregunta es si elegir el mejor sirve. Y las copias inflan el recuento de «variantes por debajo» o «por encima» de la elegida: en G, 112 de las duplicadas son del estrato factorial.

De paso, lo que sale si se trata a **la madre** (`P00000`) como si fuera la elegida en todas las particiones —su puesto mediano fuera, entre sus variantes—:

madre	puesto mediano de la madre	particiones en la mitad de abajo
G	0,96	0,1 %
F	0,87	8 %
`10.11.484`	0,83	14 %
E	0,69	29 %
`19.2.363`	0,65	16 %
C	0,63	24 %
`16.5.352`	0,62	30 %
`3.9.284`	0,60	21 %
`18.5.355`	0,59	32 %

(`13.3.390` : su madre no pasa el suelo y no está en el panel.)

En las nueve, la madre queda por encima de la mitad de sus variantes en la mayoría de las particiones; en G, por encima del 96 %. No es un PBO (no hay elección: la madre se eligió antes, con el `build`), pero es una señal de que **los parámetros originales están cerca del mejor punto, no en mitad de una meseta**. El estudio no lo calcula.

5.8 Ningún número llega a los hechos, y una regla `"18"` no se juzgaría

- `pipeline/autopilot/facts.py:221-222` sólo recoge resultados cuya ruta es una carpeta; el del CSCV es `<lote>/estudios/cscv.json` .
- `contract.py:81-82` deja `score` y `value` vacíos.
- `pipeline/autopilot/tail.py:110` apunta los pasos del bloque con `add(n, "blind", …)` ; el juez sólo se llama para el 20 y del 21 al 25 (`tail.py:93`).
- La descripción de `flat` en `report.py:52-64` dice que los escalares los lee `pipeline/stages/verdict.py` : es la cadena antigua.

Para escribir una regla sobre el PBO, la pendiente o «pierde» faltan las tres primeras cosas. Es trabajo de código, no una decisión estadística.

5.9 La frontera entre tramos no se lee de donde dicen los README

`engines/variants/panel.py:140` la lee de `assetdata.segment_start` (la ficha del activo). El README de `engines/variants` y el comentario de `cscv/config.yaml` (líneas 6-7) dicen que sale de `equity.json` del lote. En esta corrida coinciden (2020-01-01 y 2024-01-01); si la ficha cambiara después de fabricar un lote, el estudio partiría por la fecha nueva.

5.10 El DSR se calcula con el `build` dentro

`report.py:155-158` pasa la ventana de dentro (`build` + `oos1` , 3.014 días) a `trials.deflated` . La mejor variante se elige ahí y su Sharpe se mide ahí. Tu doctrina (PSR y trader al azar, sólo en OOS; nunca en el IS) no lo permite. Además:

- deflacta sólo por los grupos de variantes de **una** madre (3 a 36), no por la búsqueda del generador;
- en cuatro de diez la silueta dice que los grupos no existen (3.10);

- con cada variante como prueba, C pasa de 0,93 a 0,20 (3.11).

No entra en el veredicto. Sale en la frase de la ventana como «el Sharpe del mejor sobrevive con probabilidad…».

5.11 Cosas pequeñas del umbral y de los bloques

- Texto «por debajo del 50 %» frente a código «hasta el 50 % incluido» (3.12).
- Con N impar, la variante que cae justo en el medio (λ = 0) cuenta como sobreajustada (`summary.py:21`, `<=`).
- Bloques de 233 y 232 días; el artículo los quiere iguales (2.2).
- Los bloques 9 y 13 pisan dos tramos a la vez (1.4). Los costes de cada tramo son distintos (cada uno lleva los de su época), así que dentro de esos dos bloques hay un cambio de costes. No he rastreado los costes de cada tramo de la fábrica.

5.12 Otros textos que no coinciden con el código

- Los README de `sqx/variants/` hablan de 5.000 variantes; el tope es 500.
- `pipeline/recipe.yaml:154` llama a `cscv.report --work {work}` sin `--family`, que ahora es obligatorio (`report.py:93`). No lo he ejecutado para confirmarlo; es la cadena antigua.
- `studies/optimisation/cscv/verdict/README.md:9` dice que pasar de 252 a 924 particiones movió el PBO de Argmax del 41 % al 30 %. Aquí, de S = 10 (252) a S = 12 (924), seis madres bajan y cuatro suben; no hay una dirección clara.

5.13 Lo que no está medido

- **La curva de potencia** sobre los paneles reales: qué PBO sale cuando sí hay un orden verdadero de un tamaño dado. El revisor la tiene sobre un panel sintético (2.12); sobre los reales no está.
- Una población sin ventaja pasada de verdad por los pasos 16.5 y 18 (lo que pide el comentario de `criteria.yaml`). La nula de «variantes iguales» la cubre en lo que toca al nivel; no cubre que las variantes de una madre sin ventaja difieran por costes.
- La nula con más de 150 sorteos y cuatro madres, y la de `3.9.284`.
- El efecto de 5.1 sobre un lote de variantes.
- Si un PBO bajo aquí predice algo después: en la WFM, en `oos2` leído limpio, o en real. Con diez madres hay un indicio: las dos de PBO bajo (G y `3.9.284`) son las únicas donde el orden de las variantes en `build` + `oos1` se parece al de `oos2` (correlación de rangos +0,58 y +0,71; las otras ocho, entre −0,27 y +0,22). Eso es el mismo fenómeno medido dos veces, no una confirmación independiente.
- Por qué el plateau empeora el PBO donde difiere de Argmax.
- Cuánto tarda el paso.
- Cuántas veces se ha aplicado a mano, con `/curate`, el DESCARTAR del paso 20. El autopiloto no lo aplica (1.6).

5.14 Dónde este documento corrige o matiza al informe anterior

- **«Sólo anota»** (informe anterior, §9). Cierto para `criteria.yaml`. Pero el estudio sí emite `pass` / `fail` con un umbral, y el paso 20 lo usa con `sin_fallo`. Hay un criterio cargado; está en otro fichero.

- **«Entre 0,08 y 0,92 no se distingue del azar».** Se sostiene: con la nula sobre paneles reales el tramo del 5 al 95 % es 0,18 – 0,79, y 0,08 – 0,92 es el tramo de dos desviaciones típicas.
- Lo que no estaba: que el azar del que se habla es «variantes iguales», y que un PBO bajo certifica un orden estable entre variantes, del tamaño que sea; no afirma ni niega robustez.

---

6. Cierre

6.1 Las reglas, en una tabla

dónde	regla	hecho	`pass`	Limbo	`fail`
`criteria.yaml`, paso 18	—	—	todas	—	—
`cscv/contract.py:87`	PBO Argmax	(no es un hecho; va en el estado del veredicto)	≤ 0,5	no existe	> 0,5
`blindJoint`, paso 20	`sin_fallo`	estado de cada pieza	ninguna pieza en `fail`	—	alguna en `fail` → DESCARTAR en `verdict.csv`; el autopiloto no lo aplica
`criteria.yaml`, paso 20	—	—	todas	—	—

6.2 El resultado sobre la población real

	`pass`	`fail`
CSCV, diez madres	**8**	**2**

En el paso 20 (`AlgoData/reports/Trade_NIKKEI225_M30_volatilidadL/WFM/2026-10-04/blindJoint/verdict.csv`): las diez DESCARTAR, «lectura sin_fallo+entrantes: No pasa». `marketSurfaces` falla en las diez. El WFC falla en seis (`13.3.390`, C, F, `16.5.352`, `18.5.355`, E), y las dos que suspenden el CSCV están entre esas seis: **hoy el CSCV no descarta a ninguna que otra pieza no descarte ya.**

Las lecturas de este documento, una al lado de otra:

madre	PBO	estado	¿fuera de la nula?	pendiente	pierde	elegida: Sharpe dentro → fuera	DSR
G `4.13.338`	0,073	pass	p < 0,02 sin corregir	+0,56	0,2 %	1,09 → 0,77	0,98
`3.9.284`	0,140	pass	indicio (bajo el p5 conjunto)	+0,74	2,1 %	1,06 → 0,68	0,96
C `14.3.479`	0,292	pass	PBO no; pendiente sí	+0,60	10,2 %	0,77 → 0,38	0,93
`18.5.355`	0,346	pass	no	+0,43	4,2 %	0,93 → 0,40	0,99
F `15.9.352`	0,350	pass	no	+0,13	54,5 %	0,47 → −0,05	0,55
`10.11.484`	0,385	pass	no	+0,30	3,1 %	0,97 → 0,46	0,99
`19.2.363`	0,391	pass	no	+0,23	51,3 %	0,66 → 0,00	0,94
`16.5.352`	0,453	pass	no	+0,06	33,8 %	0,77 → 0,11	0,93
`13.3.390`	0,642	**fail**	no	−0,16	26,2 %	0,69 → 0,15	0,96
E `3.11.426`	0,653	**fail**	no	−0,09	7,0 %	1,00 → 0,37	0,93

(«¿Fuera de la nula?» para las madres sin sorteo propio se lee contra los percentiles conjuntos de C, E, F y G. «No» quiere decir «no rechaza la nula», con poca potencia; no «no hay orden».)

6.3 Las decisiones que te quedan

Cada una con su coste en estrategias (sobre las diez) junto a lo que gana.

1. **¿Qué pregunta quieres que conteste este paso?** Es la decisión de la que cuelgan las demás. (a) «¿Elegir parámetros sirve en esta estrategia?»: el PBO la contesta a medias: G da un sí (p < 0,02 sin corregir), `3.9.284` un indicio, y de las otras ocho no se puede decir ni sí ni no. (b) «¿Es una meseta?»: el PBO no la contesta; una meseta exacta da 0,5 y una con diferencias minúsculas pero ordenadas puede dar 0,1. (c) «¿Cuánto infla el backtest elegir el mejor punto?»: la contesta la caída de Sharpe de la elegida (3.4), que el estudio no guarda. Coste de decidir: 0.

2. **¿Dejar el `fail` en PBO > 0,5?** Coste: **2 de 10** marcadas (`13.3.390` , E), 0 borradas hoy (el autopiloto no corta en el 20, y las dos caen también por el WFC). Ganancia: ninguna que se pueda defender; la mitad de las mesetas perfectas lo suspenden. Lo que no sabes: cuántas estrategias sin ventaja lo aprueban.

3. **¿Subir la frontera de `fail` a donde el azar casi no llega (0,8; el 4,2 % ± 1,6 de los nulos)?** Coste: **0 de 10**. Ganancia: el `fail` pasaría a significar algo («el mejor de dentro tiende a ser de los peores de fuera»), al precio de no dispararse casi nunca. Umbral sin calibrar fuera de mis 600 sorteos.

4. **¿Quitarle el `pass` / `fail` y dejarlo en anotación de verdad?** Coste: 0. Con `sin_fallo` , una pieza que nunca falla no pesa. Es coherente con el comentario de `criteria.yaml` , que hoy el código no cumple del todo.

5. **¿Añadir un tercer estado?** Hoy no hay Limbo. La lectura que sale de la nula sería: por debajo de 0,18, «orden estable»; entre 0,18 y 0,8, «no se distingue»; por encima, «elegir perjudica». Sobre las diez: **2 · 8 · 0**. No lo propongo como regla: lo que falta para escribirla es lo del punto 9.

6. **¿Sacar los controles y las copias del panel?** Coste en estrategias: 0 hoy (ninguna cruza 0,5: `19.2.363` sube a 0,441, G a 0,121). Ganancia: que el PBO responda a «elegir entre variantes de verdad»; los movimientos medidos (0,03-0,05) están dentro del ruido. Es una decisión de qué es el conjunto, y cambia el número: mirada nueva.

7. **¿Qué suelo y qué reparto?** Hoy el suelo de 50 en `oos2` quita del 9 % al 36 % del panel y decide el veredicto de una madre (`13.3.390`: `fail` con suelo, `pass` sin él). Opciones: dejarlo (coste: 2 fallan), quitarlo del CSCV y dejarlo sólo para el bloque cronológico (1 falla), o corregir el comentario que dice que no influye. No sé cuál es mejor: sin suelo entran variantes que casi no operan en `oos2`, y su Sharpe en los bloques 14-16 es ruido.

8. **¿Qué hacer con el plateau, la dominancia, `drift` y el DSR?** Ninguno decide hoy. Plateau: o se arregla la vecindad o se quita (como primera regla costaría **4 de 10**). Dominancia: constante; se puede dejar de escribir. `drift`: la unidad no es comparable entre madres. DSR: decidir si se calcula sobre `oos1` + `oos2` o se retira, por tu doctrina. Coste en estrategias de las cuatro: 0.

9. **¿Exponer hechos y correr la prueba de vacuidad?** Lo primero es código (5.8). Lo segundo es lo que pide `criteria.yaml`: una población sin ventaja por los pasos 16.5 y 18, y contar cuántas sacan PBO bajo. Coste: una corrida de la fábrica de variantes por cada tanda. Ganancia: la única cifra que convierte un umbral de PBO en algo más que una convención. La nula del apartado 2.12 es más barata (minutos, sin SQX) y ya cubre «sin ventaja con variantes iguales», porque el PBO no depende del nivel común. Lo que falta por ese camino barato es la curva de potencia: qué PBO da un orden verdadero de cada tamaño.

10. **¿Reconstruir las celdas como ganancia diaria de verdad?** Pide las operaciones de cada variante. Hasta entonces, todos los números de este paso llevan encima un error de tamaño desconocido que en poblaciones llegó a 0,12 de PBO.

6.4 El límite que no cambia

El CSCV contesta una pregunta estrecha: **dentro de este conjunto de variantes y de este histórico, ¿el orden entre ellas se mantiene?** La contesta con un número que baila 0,19 arriba o abajo cuando no hay nada que medir, así que sólo habla claro en los extremos, y en esta corrida habla de una madre de diez ($p < 0{,}02$ sin corregir) y deja un indicio de otra. No dice si la estrategia gana, no dice si es una meseta —mira el orden entre variantes, no el tamaño de sus diferencias—, y no dice nada de cómo se llegó a la madre. Lo que sí deja, y hoy no se guarda, es la cifra más honesta del paso: cuánto Sharpe pierde el mejor punto cuando se le cambia el trozo de histórico. En la mejor de las diez, un tercio; en dos de las que aprueban, todo.